

Save the summary content in markdown format

markdown_content = """

Major Differences between Deep Learning (DL) and Machine Learning (ML)

1. Definition & Scope

- **Machine Learning:** A broad field of AI where algorithms learn from data to make predictions or decisions.
- **Deep Learning:** A subset of ML using deep neural networks with multiple layers to learn hierarchical features automatically.

2. Model Architecture

- **ML models:** Include linear regression, decision trees, and random forests; usually require manual feature design.
- **DL models:** Built with many stacked layers (e.g., CNNs, RNNs, transformers); learn features end-to-end via backpropagation.

3. Feature Engineering & Data Requirements

- **Classical ML:** Requires human-engineered features and structured, labeled data.
- **Deep Learning:** Works directly with raw, unstructured data (e.g., text, images) and learns features automatically.

4. Computational Cost & Scalability

- **ML algorithms:** Less computationally intensive; run efficiently on CPUs for small to medium datasets.

- **DL algorithms:** Require high-performance GPUs or distributed computing due to large model complexity and data needs.

5. Interpretability & “Black-Box” Nature

- **ML:** Models like linear regression and decision trees are more interpretable.
- **DL:** Neural networks are often opaque, making explainability more challenging.

6. Peak Performance & Typical Use Cases

- **ML:** Best suited for smaller datasets and tasks requiring explainability (e.g., fraud detection, predictive modeling).
- **DL:** Outperforms ML on large-scale problems like image recognition, speech processing, NLP, and generative AI.

In short: Deep Learning is “scalable, feature-learning, multi-layer” Machine Learning optimized for big, unstructured data at high computational cost, while classical ML covers a broader set of lighter-weight, often more interpretable algorithms suited to structured data and smaller problems.

"""

Save to markdown file

```
md_path = "/mnt/data/dl_vs_ml_summary.md"
```

```
with open(md_path, "w") as f:
```

```
    f.write(markdown_content)
```

```
md_path
```

Week 5: Supervised Machine Learning Model I

1. Machine Learning Overview

- **Definition:** Algorithms that learn from data to make predictions on unseen inputs.
- **Paradigms:**
 - **Supervised:** Classification, Regression, Time Series
 - **Unsupervised:** Clustering, Dimension Reduction, Association Analysis

2. Regression

- **Simple Linear Regression**
 - Model: $(Y = \beta_1 X + \beta_0)$
 - Fit via Ordinary Least Squares (minimize SSE)
 - Key concepts: covariance, correlation coefficient $(-1 \leq \rho \leq 1)$
- **Multiple Linear Regression**
 - Model: $(Y = \beta_1 X_1 + \dots + \beta_n X_n + \beta_0)$
 - OLS extended to multiple predictors
 - Evaluation:
 - **R²** measures explained variance
 - **Adjusted R²** penalizes adding useless predictors to avoid overfitting

3. Optimization: Gradient Descent

- **Cost Function:** $J(\theta) = \frac{1}{2N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i)^2$
- **Variants:**
 - **Batch GD** (all data per update)
 - **Stochastic GD** (one example per update)

- **Mini-Batch GD** (small batches per update)
- **Update Rule:**

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{N} \sum_{i=1}^N (h(\theta) - y_i) x_{ij}$$
- Illustrated with 3D cost surface, convergence plots, and Python code examples :contentReference[oaicite:3]{index=3}

4. Classification: Logistic Regression

- **Model:**

$$P(Y=1 \mid X) = \frac{1}{1 + e^{-z}}, \quad z = \alpha + \sum_{i=1}^k \beta_i X_i$$

Decision threshold at 0.5
- **Parameter Estimation:** Maximum Likelihood via gradient ascent
- **Evaluation Metrics:**
 - **McFadden's R²** (pseudo-R²)
 - **AIC** to compare models and guard against overfitting :contentReference[oaicite:4]{index=4}
- **Hands-On:** Titanic dataset ROC curve and scikit-learn implementation

5. Tools & Workflow

1. **Data Assessment:** scatter plots, correlations
 2. **Model Fitting:** `LinearRegression`, `LogisticRegression` (scikit-learn)
 3. **Validation & Evaluation:** R², adjusted R², ROC/AUC, Confusion Matrix
 4. **Deployment:** select model balancing bias-variance and interpretability :contentReference[oaicite:5]{index=5}
-

This summary distills the key definitions, models, algorithms, and evaluation techniques from the Week 5 lecture.

Week 6: Supervised Machine Learning Model II

1. Decision Trees

- **Concept & Structure**
 - Tree induction builds a flowchart of if-then tests, splitting data into homogeneous subgroups (root, internal, leaf nodes).
- **Splitting Criteria**
 - **Gini Index** (CART) and **Entropy / Information Gain** (ID3) measure node “purity.”
 - Choose splits by minimizing the weighted sum of child-node impurities.
- **Tree Growth & Pruning**
 - Grow a large tree, then prune using a cost-complexity parameter (cp) and cross-validation.
 - The “1 SE rule” picks the simplest tree within one standard deviation of minimum error.

2. k-Nearest Neighbors (k-NN)

- **Principle**
 - “Memorize” the training set; classify new points by majority vote among their k closest neighbors.
- **Distance Metrics**

- Euclidean, Manhattan, Chebyshev, plus similarity measures like correlation or cosine.
- **Parameter Selection**
 - Pick an odd k for binary problems; tune k and distance metric via validation (e.g., plot accuracy vs. k).

3. Support Vector Machines (SVM)

- **Linear SVM**
 - Seeks a hyperplane that separates classes, maximizes margin, and can ignore outliers (soft margin).
 - Formulated as a quadratic program: minimize $\|\beta\|^2$ subject to $y_i(\beta^T x_i + \beta_0) \geq 1$.
- **Non-Linear SVM**
 - Applies the kernel trick (e.g., polynomial, RBF) to map data into higher dimensions for linear separation.
- **Modeling Workflow**
 1. Preprocess (e.g., scale features)
 2. Choose kernel (start linear, then RBF)
 3. Tune hyperparameters via grid/randomized search and cross-validation
 4. Evaluate on test set before deployment

4. Naïve Bayes

- **Bayes' Theorem**

$$P(Y \mid X) = \frac{P(Y) \prod_i P(X_i \mid Y)}{P(X)}$$
- **Conditional Independence** assumption: features X_i are independent given class Y .
- **Variants**
 - **Multinomial NB** for count data

- **Gaussian NB** for continuous features
- **Example:** Predict “Play Golf” from weather attributes by computing priors and likelihoods.

5. Model Development & Evaluation Workflow

1. Data Split

- Train / validation / test (e.g., 80 / 10 / 10) or k-fold CV + test.

2. Model Training

- Train multiple candidates on the training set.

3. Validation

- Select the best model & hyperparameters using validation or CV.

4. Testing

- Evaluate final model on the held-out test set.

6. Performance Metrics & Diagnostics

- **Confusion Matrix**

- TP (True Positives), TN, FP, FN

- **Basic Metrics**

- **Accuracy** = $(TP + TN) / \text{Total}$
- **Precision** = $TP / (TP + FP)$
- **Recall / Sensitivity** = $TP / (TP + FN)$
- **Specificity** = $TN / (TN + FP)$
- **Negative Predictive Value (NPV)** = $TN / (TN + FN)$

- **Composite Metrics**

- **F1 Score** = $2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$
- **Matthews Correlation Coefficient (MCC)**, robust for imbalanced data

- **ROC & AUC**

- Plot True Positive Rate vs. False Positive Rate; AUC measures overall separability.

This markdown captures the key algorithms, workflows, and evaluation techniques from the Week 6 slides on Supervised Machine Learning Model II.

Week 7: Deep Learning I

1. Artificial Neural Networks (ANN)

- **Inspiration:** Modeled after biological neurons—interconnected processing units working in parallel.
- **Artificial Neuron:**
 - Computes a weighted sum of inputs + bias, then applies an activation (step, sigmoid, ReLU, etc.).
 - **Perceptron** is the simplest unit with a step activation. :contentReference[oaicite:0]{index=0}

2. Hands-On Examples

- **Perceptron on MNIST:** Classify 28×28 handwritten digits with a single-layer perceptron.
- **Multilayer Perceptron (MLP):**
 - Flexible hidden-layer sizes, activations (`identity` , `logistic` , `tanh` , `relu`), solvers (`sgd` , `adam`), learning rates.
 - Applied to both classification and regression tasks (e.g., yacht resistance). :contentReference[oaicite:1]{index=1}

3. ANN Architectures

- **Feedforward:** Data flows input → hidden → output, no cycles.

- **Recurrent (Feedback):** Includes loops to maintain state across steps.

4. Model Development & Learning

1. Collect & preprocess data
2. Split into train/validation/test
3. Choose architecture & initialize weights
4. Train via forward pass + backpropagation (e.g., SGD, Adam)
5. Monitor cost (MSE) & apply early stopping
6. Evaluate on test set before deployment

5. Overfitting & Capacity

- **Early Stopping:** Halt when validation error stops improving.
- **Hidden-Layer Sizing Rules** (guidelines only):
 - $\sim 2/3$ of input size, or average of input/output sizes.
 - Adjust based on data complexity & activation functions.

6. Perceptron Learning Algorithm

- **Weight Update:**

$$(\Delta w_i = \alpha (y_{\text{true}} - y_{\text{pred}}) x_i)$$
- Demonstrated on logical OR, AND, XOR (XOR requires MLP).

7. Deep Learning Foundations

- **Depth:** ≥ 2 hidden layers $\rightarrow$ "deep."
- **Tensors:**
 - 1D vectors, 2D images, 3D color, 4D video.

- **Bias Units:** Enable nonzero output when inputs are zero.
- **Universal Approximation:** Two hidden layers (nonlinear + linear) can approximate any continuous function. :contentReference[oaicite:6]{index=6}

8. Activation Functions

- **Hidden Layers:** ReLU default; alternatives include sigmoid, tanh, softplus.
- **Output Layer:**
 - Regression → linear
 - Binary classification → sigmoid
 - Multiclass → softmax :contentReference[oaicite:7]{index=7}

9. Computational Graphs

- Represent networks as dataflow graphs; underpin automatic differentiation & efficient backpropagation. :contentReference[oaicite:8]{index=8}

10. Depth vs. Width

- **Deep & Narrow** networks often outperform **shallow & wide** for learning hierarchical features.
- Modern frameworks provide automatic differentiation and optimizers like Adam. :contentReference[oaicite:9]{index=9}

11. Beyond MLPs

- Preview of other DL architectures: **CNNs**, **RNNs**, **Autoencoders**, etc. :contentReference[oaicite:10]{index=10}

Week 8: Deep Learning II

1. Deep Learning Architectures

- **Convolutional Neural Networks (CNNs)** for spatial data (images, video, even time-series)
- **Recurrent Neural Networks (RNNs)** for sequential/temporal data, with hidden-state feedback loops
- **Autoencoders** for unsupervised representation learning (dimensionality reduction, denoising)
- **...and more** (e.g., GANs, Transformer variants) :contentReference[oaicite:1]{index=1}

2. Advanced CNN Concepts

- **Convolution & Feature Maps**
 - Convolution (*) extracts local patterns via learnable filters → stacked into multi-channel feature maps
 - Multiple filters (e.g., horizontal/vertical edge detectors) produce a volume of feature maps
- **Padding (p) & Stride (s)**
 - **Padding** adds dummy pixels to preserve spatial dimensions
 - **Stride** controls the step size of the filter, downsampling feature maps
- **Pooling**
 - **Max-pooling** retains the strongest activation (dominant feature)
 - **Average-pooling** computes local averages
 - **Sum-pooling, min-pooling**, etc., chosen based on data characteristics

- **Dimensionality Formula**

$$\begin{aligned} &[\\ &\text{output_dim} = \frac{n + 2p - f}{s} + 1 \\ &] \end{aligned}$$

where (n)=input size, (f)=filter size, (p)=padding, (s)=stride :contentReference[oaicite:2]{index=2}

3. Iconic CNNs & Visualizations

- **LeNet-5** for handwritten digit recognition ($28 \times 28 \rightarrow$ softmax over 10 classes)
- **AlexNet (2012 ImageNet Winner)**
 - 1.2 M images, 1 000 classes $\rightarrow$ top-5 error reduced from 26 % $\rightarrow$ 15.3 %
 - Visualized first-layer filters and deeper feature maps to show hierarchy of learned patterns :contentReference[oaicite:3]{index=3}

4. Improving & Regularizing Networks

- **Architecture Search**
 - Deeper networks often outperform shallow ones (e.g., 11-layer $>$ 3-layer)
 - Automated search: genetic algorithms, hill climbing, reinforcement learning
- **Penalizing Large Weights** (L2 regularization) to discourage over-complex models
- **Dropout**
 - Randomly “drops” units during training to reduce co-adaptation and overfitting :contentReference[oaicite:4]{index=4}

5. Recurrent Neural Networks (RNNs)

- **Definition:** Networks with cycles in their computational graphs, carrying internal state (memory) across time steps
- **Unfolding:** Treat an RNN as a sequence of tied-weight networks over time
- **RNN Variants & Use Cases**
 - **One-to-one:** classic feedforward behavior (e.g., static image classification)
 - **One-to-many:** single input $\rightarrow$ sequence output (e.g., image captioning)
 - **Many-to-one:** sequence input $\rightarrow$ single output (e.g., sentiment analysis)
 - **Many-to-many:** sequence input $\rightarrow$ sequence output (e.g., machine translation)

- **many-to-many**. sequence input ↔ sequence output (e.g., machine translation, video action recognition) :contentReference[oaicite:5]{index=5}

6. Autoencoders

- **Encoder-Decoder** architecture learns a compressed representation, then reconstructs input
- **Unsupervised Pre-training**: Stacked denoising autoencoders for feature learning and noise removal
- **Use Cases**: Dimensionality reduction (akin to PCA), generative modeling, anomaly detection :contentReference[oaicite:6]{index=6}

7. Transfer Learning

- **Concept**: Reuse weights from a pre-trained network on Task A to accelerate/boost learning on Task B
- **Techniques**:
 - **Feature extraction**: freeze convolutional base, train new classifier layers
 - **Fine-tuning**: unfreeze top layers of the base and jointly train with new layers
- **Beyond Vision**: Pre-trained NLP models (e.g., RoBERTa) adapted to domain-specific vocabulary and tasks :contentReference[oaicite:7]{index=7}

8. Application Domains

- **Computer Vision**: robotics, self-driving cars, healthcare imaging, manufacturing, agriculture
- **Natural Language Processing**: sentiment analysis, entity recognition, summarization, translation
- **Reinforcement Learning**: policy optimization via reward signals (traffic control, games, resource management) :contentReference[oaicite:8]{index=8}

9. Deep Learning Frameworks

- **Libraries & Tools**: H2O, Theano, Lasagne, MxNet, Caffe, TensorFlow

- **LIBRARIES & TOOLS.** N2O, Theano, Lasagne, NOLearn, ConvNets, Caffe, TensorFlow, Keras, Chainer, PyTorch :contentReference[oaicite:9]{index=9}